Introduction

Frame-It offers a straightforward way to define and use custom environments in your documents. Its syntax is designed to integrate seamlessly with your source code.

Two predefined styles are included by default. You can also create custom styling functions that use the same user-facing API while giving you complete control over the Typst elements in your document.

Distinct Highlight | Best for occasional use | More noticeable

Feature 1

The default style, `styles.boxy`, is eye-catching and intended to stand out from the surrounding text.

In contrast:

> Feature 2 **Unobtrusive Style** *(Ideal for frequent use, Blends into text flow)* : The alternative style `styles.hint` highlights text with a subtle colored line along the side, preserving the document's flow.

> **Elegant** Separates header and content not too obtrusive **Feature 3**
>
> The third default style `styles.hint` clearly separates frame header and frame body. This style was taken from [typst-thmbox](https://github.com/s15n/typst-thmbox).

The default styles are merely functions with the correct signature. If they don't appeal to you, you have complete freedom to define custom styling functions yourself. For reference, this is that signature:

```
let your-custom-styling-function(title, tags, body,
supplement, number, accent-color) = […]
```

A different frame kind

Example 4

You can define different classes or types of frames, which alter the substitute and the frame's color. As shown here, this is an example frame. You can create as many different kinds as you want.

As long as all kinds use the same kind, they share a common counter.

Quick Start

Import and define your desired frames:

```
#import "@preview/frame-it:1.2.0": *

#let (example, feature, variant, syntax) = frames(
  feature: ("Feature",),
  // For each frame kind, you have to provide its supplement
title to be displayed
  variant: ("Variant",),
  // You can provide a color or leave it out and it will be
generated
  example: ("Example", gray),
)
// This syntax works as well, but colors are not generated
automatically
#let syntax = frame("Syntax", green)
// This is necessary. Don't forget this!
#show: frame-style(styles.boxy)
```

How to use it is explained below. Here is a quick example:

```
#example[Title][Optional Tag][
  Body, i.e. large content block for the frame.
]
```

which yields

Title Optional Tag

Example 5

Body, i.e. large content block for the frame.

Feature List

The following features are demonstrated in all predefined styles.

Seamlessly hightight parts of your document

Feature 6 **Element with Title and Content** : The simplest way to create an element is by providing a title as the first argument and content as the second.

Feature Variant 7 **Element with Tags** *(Customizable Tags, Multiple)* : Elements can include multiple tags placed between the title and the content.

Feature 8 : If you don't require a custom title but still want to display the element type, use `[]` as the title placeholder.

Feature Variant 9 *(Single Tag, Next tag)* : You can include tags even when no title is provided.

To omit the header entirely, leave the title parameter empty.

Feature 11 **Element without Content** *(Optional Tags Only)*

For brief elements, use [] as the body to omit the content.

Feature 12 **Element with Divider** : Insert `#divide()` to add a divider within your content for a visual break:

And then continue with your text below the divider.

Highlight parts distinctively

Element with Title and Content

Feature 13

The simplest way to create an element is by providing a title as the first argument and content as the second.

Element with Tags | Customizable Tags | Multiple

Feature Variant 14

Elements can include multiple tags placed between the title and the content.

Feature 15

If you don't require a custom title but still want to display the element type, use `[]` as the title placeholder.

Single Tag | Next tag

Feature Variant 16

You can include tags even when no title is provided.

To omit the header entirely, leave the title parameter empty.

Element without Content | Optional Tags Only

Feature 18

For brief elements, use [] as the body to omit the content.

Element with Divider

Feature 19

Insert `#divide()` to add a divider within your content for a visual break:

And then continue with your text below the divider.

A third Alternative

We recently added third style, namely `styles.thmbox`:

Element with Title and Content **Feature 20**

The simplest way to create an element is by providing a title as the first argument and content as the second.

Element with Tags Customizable Tags Multiple **Feature Variant 21**

Elements can include multiple tags placed between the title and the content.

Feature 22

If you don't require a custom title but still want to display the element type, use `[]` as the title placeholder.

Single Tag Next tag **Feature Variant 23**

You can include tags even when no title is provided.

To omit the header entirely, leave the title parameter empty.

Element without Content Optional Tags Only **Feature 25**

For brief elements, use [] as the body to omit the content.

Element with Divider **Feature 26**

Insert `#divide()` to add a divider within your content for a visual break:

And then continue with your text below the divider.

Miscellaneous

Syntax to create single frames

Single–Frame–Creation

Syntax 27

When the above method for creating frames is not flexible enough, you can use this alternative method for creating frames one–by–one:

```
#let theorem = frame("Theorem", red)
#let definition = frame(kind: "not-the-default",
"Definition", blue)
```

Contrary to the first method, you have to specify a color.

HTML

We were one of the first packages to add support for html!
This means you can use our frames if you're writing an html wiki or blog in typst.

HTML export

Feature 28

Due to the currently experimental nature of the feature, you need to pass two CLI–flags when compiling your document with the frames to html:

```
typst compile --features html --format html FILE.typ
```

General

Internally, every frame is just a `figure` where the `kind` is set to `"frame"` (or a different custom value). As such, most things that can be done to a figure can be done with a frame as well. Whenever you would like to do something custom but don't know if it is supported, try achieving it with a normal figure first and then apply the same show rule to your frames. Here is a list of examples:

Labels and References

Feature Variant 29

Elements can be referenced as expected by appending `<label>` and referencing it:

```
#syntax[Labels and References] <labels-and-refs>
Referencing with @labels-and-refs.
```

Break frames across pages

Feature Variant 30

If you want to make your frames breakable across pages, you have to use the show rule known from the official typst docs:

```
#show figure.where(kind: "frame"): set block(breakable:
true)
```

To turn off breakability, you can use the corresponding show rule

```
#show figure.where(kind: "frame"): set block(breakable:
false)
```

Per–frame custom figure parameters

Frame outline

Feature 31

All named parameters passed to a frame–function like `example[][]` are going to be passed to the figure function which places the frame in the document.

For example, if you want to give a frame a specific number, you can achieve this with

```
#example()[Frame without number][This frame has no number.]
```

For example, you can create an outline which only contains some intentional of your frames like so. The `figure` function includes a parameter for including a figure in the outline.

```
// By default, don't include frames in outlines by default
#show figure.where(kind: "frame"): set figure(outlined: false)
// Create the outline
#outline(target: figure.where(kind: "frame"))
// Explicitly include a frame in the outline with the
```

```
`outlined` parameter.
#example(outlined: true)[Important frame][For the outline]
```

Different numbering Feature Variant 32

Numbering in figures is a bit of a mess. Natively, you are limited to one number and a format suffix/prefix. With that, you can do the following.

```
#show figure.where(kind: "frame"): set figure(numbering: "a)")
```

If you want to do more advanced and nested numbering, you can look into external packages for that. When trying to make a system apply to the frames, remember that they are just figures with a specific kind.

If you don't want to display the number at all, you can pass instead.

In the background, there are also some show rules doing the styling for the frames. However, these should only get in the way when you are doing some esoteric manipulation of the figure captions.

Different kind for the underlying figures Syntax 33

When you have multiple different groups of frames you need to style independently, you can provide an arbitrary `kind` to the `frames` function.

```
#let (syntax,) = frames(
  syntax: ("Syntax",),
)
#let (note,) = frames(
  kind: "other-kind"
  note: ("Note",),
)
#show: frame-style(styles.boxy)
#show: frame-style(kind: "other-kind", styles.hint)
```

Change the styling

Syntax 34

To use a different styling function for just one frame, you can provide `style: some-style` as an extra argument:

```
#variant(style: styles.hint)[
  This has hint styling
]
```

When you want to change the styling used for a passage of your document, you can just add more `show: frame-style()` rules:

```
#show: frame-style(styles.boxy)
#example[In boxy style][]
#show: frame-style(styles.hint)
#example[In hint Style][]
```

I usually define the abbreviations for the show rule:

```
// Define once
#let boxy(document) = {show: frame-style(styles.boxy);
document}
#let hint(document) = {show: frame-style(styles.hint);
document}

// Use it for changing the style used
#show: boxy
#example[In boxy style]
#example[Also in boxy style]
#show: hint
#example[In hint style]
```

Custom Styling

Internally, there is nothing special about the predefined styles. The only requirement for any styling function is to adhere to the following function signature interface:

Interface for custom styling function

Syntax 35

```
#let custom-styling(title, tags, body, supplement, number,
arg)
```

where `arg` is going to be the value passed behind the supplement for each frame variant in the `frames` function. For the predefined styles, this is the color of the frames. When defining your own styling function, it has to have the following signature:

The content returned will be placed as–is in the document.

Styling Dividers

Syntax 36

If your custom styling function shall support dividers, it must include this show rule in its body:

```
#show: styling.dividers-as(object-which-will-be-used-as-divider)
```

For more information on how to define your own styling function, please look into the `styling` module.

`frame-it` combines very well with [showybox](https://typst.app/universe/package/showybox/) in order to create beautiful styles easily!

Experimentl APIs

These APIs are still experimental and subject to change. Use sparingly.

Retrieving information of a frame Experimental

Syntax 37

When you want to do more intricate styling for your frames, sometimes it is necessary to obtain all the information which.

For this, use the `inspect.lookup-frame-info(figure)` function. Given a figure (which represents a frame), it returns a dictionary with the entries `title`, `tags`, `body`, `supplement`, `color`.

When the provided figure is not a frame, this function throws an error.

For example, you can use this to color the entries in a outline according to the color of the frame:

```
#show outline.entry: it => {
  let color = inspect.lookup-frame-info(it.element).color
```

```
  text(fill: color.saturate(70%), it)
}
#outline(target: figure.where(kind: "frame"))
```

Checking whether a figure is a frame | Experimental

Syntax 38

In order to test whether a figure is a frame, use the `inspect.is-frame(figure)` function.

Whenever possible, try to discern it using the figures kind instead of this function.

This can be useful, for example, when you want to format references in your document differently. For example, you can do something like this in order to display just title if the referenced element and link to it:

```
#show ref: it => {
  if inspect.is-frame(it.element) {
    link(it.element.location(), inspect.lookup-frame-
info(it.element).title)
  } else {
    it
  }
}
```

Edge Cases

Here are a few edge cases.

Example 39

Test	Long tag example without space for the supplement	notice the number moves up

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

Example 40

Example | Tags of various sizes | $\sum_{\sum}^{\sum}$ | Extra vertical space:

(Leads to formatting errors in html export because support is missing) Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

Nested

Example 41

Example 42

Example 43 : When nested, counters increment from outer to inner elements.

Example 44

Counters continue incrementing sequentially in non-nested elements.

Should be thmbox **Other Frame 1**

thmbox

Should be boxy

Example 45

boxy

Example 46 **Should be hint** : This used an ad–hoc style argument

No numbering

Example 47

Here, `numbering = none` is being passed.

Individual

Some Additional Frame 48

Individual

Some Additional Frame 1 **Individual** : Individual

Example 49

Tag Title

Right to left by language

Example 50

Tag Title

Right to left by dir